État d'implémentation face à la gouvernance

> ***Statut : constat.*** *Ce document décrit ce qui a été vérifié dans le code, avec ses sources. Les objectifs et les plans sont dans `cible/`.*

> *Responsable : Cédric · Version 0.4 du 21/10/2026 · Relevé sur `main` après la fusion des PR #14 à #21, #24, #26 (moteur de remédiation, détection ML, tableau de bord), #29 (Pay&Go déplacé dans `services/payment-api/`), #47 à #53 et #55 (consigne additionnelle 1, tableaux Grafana, détecteur réentraîné)*
>
> *Les documents de gouvernance ont été écrits avant le code. Ce registre dit, pour chaque hypothèse importante, si le code la **confirme**, la **contredit** ou ne l'a **pas encore implémentée**. Il est à mettre à jour après chaque fusion importante et avant les essais (WOR-73).*

1. Ce qui existe au 21/10/2026

Composant	État	Où
API de paiement (Node.js) : sessions, idempotence des paiements, métriques Prometheus, journaux JSON, limitation à 60 requêtes par minute et par client, 12 essais de connexion par 15 min, **verrouillage de l'identifiant après 2 échecs** (5 min, puis 10, 15…)	Implémentée. **Dans le socle, elle garde tout en mémoire** (voir §2)	`services/payment-api/backend/`, `infra/docker-compose.yml`
Front web et page d'administration	Implémentés	`services/payment-api/front/`
Base PostgreSQL de l'application	Implémentée, **sans sauvegarde**, utilisée seulement par le déploiement autonome de Pay&Go	`services/payment-api/infra/docker-compose.yml`
Socle : Vault, PostgreSQL, Prometheus (collecte toutes les 5 s, moteur de remédiation compris), Loki (rétention 7 jours), Grafana	Implémenté ; l'API y est collectée	`infra/docker-compose.yml`
WAF ModSecurity + OWASP CRS, en mode bloquant, seul point d'entrée publié	Implémenté, l'API est derrière lui. Injections bloquées en 403. Liste d'IP bloquées rechargée à chaud (`nginx -s reload`, 20 s maximum) ; les autres règles demandent un redémarrage	`infra/waf/`
Détection ML : Isolation Forest par famille (`traffic` → déni de service, `injection` → injection), fenêtre de 60 s au pas de 10 s, alerte après **3 fenêtres anormales**	Implémentée, en conteneur. Réentraînée le 21/10 sur 7,3 h de trafic légitime de la plateforme : environ une fausse alerte de trafic toutes les 7 h, aucune d'injection en validation. **Taux et délai de détection des attaques A et B pas encore mesurés**	`ml/inference/`, `ml/training/README.md`
Moteur de remédiation : reçoit les alertes signées, exécute une liste fermée de playbooks Ansible, vérifie par une transaction de test. **Ignore les nouvelles alertes pendant 180 s** après un redémarrage ou une restauration	Implémenté, **sur l'hôte** (pas en conteneur), avec les droits Docker de l'utilisateur	`services/remediation/`, `scripts/start-remediation.sh`
Playbooks de remédiation : blocage d'IP au WAF, redémarrage, restauration (recréation du conteneur), levée des blocages	Implémentés	`ansible/playbooks/remediation/`

Composant	État	Où
Tableau de bord d'incident (WOR-40), servi par le moteur	Implémenté	`services/dashboard/`
Tableaux de bord Grafana : SOC / SIEM, plateforme et paiements, détection et reprise (chronologie, durée de reprise, escalades)	Implémentés ; aucune règle d'alerte Grafana	`infra/grafana/provisioning/`
Remise à l'état sain	Implémentée ; **efface base, journaux, métriques et Vault**, lève les blocages	`scripts/reset-known-good.sh`
Export des journaux de Loki vers un dossier de preuves (empreintes SHA-256)	Implémenté, **à lancer à la main** : ni la remise à l'état sain ni le moteur ne l'appellent	`scripts/export-logs.sh`
Durcissement des conteneurs (non-root, `cap_drop`, limites de ressources)	Implémenté	`infra/docker-compose.yml`
Double authentification du back-office (consigne additionnelle 1) : identifiant, puis visage par la caméra, puis mot de passe ; enrôlement par l'administrateur	Implémentée (WOR-95 à 97). Exigée seulement pour les comptes dont le visage est enrôlé ; 2 échecs de visage verrouillent l'identifiant. Empreintes faciales conservées dans le volume `face-data`, jamais les images	`services/face-recognition/`, `services/payment-api/`
Objet connecté : LED de l'ESP8266 qui reflète l'état de la connexion au back-office (consigne additionnelle 1)	Implémenté (WOR-99, 100). Lit `/v1/device/led`, **ouvert si `DEVICE_TOKEN` n'est pas défini**, ce qui est le cas dans le socle	`iot/led-indicator/`
Assistant RAG en lecture seule	Implémenté	`rag/docker-compose.rag.yml`
Scripts d'attaque A et B	Implémentés ; **l'attaque B vise encore une route inexistante** (voir §3)	`attack/`

La chaîne attaque → détection → remédiation → vérification existe maintenant de bout en bout dans le socle. Elle n'a pas encore été chronométrée.

2. Hypothèses de la gouvernance confrontées au code

Point principal : dans le socle, l'API garde ses données en mémoire. Elle n'utilise PostgreSQL que si `STORAGE_DRIVER=postgres` (`services/payment-api/backend/api/src/main.ts`), variable absente pour `payment-api` dans `infra/docker-compose.yml`. Le playbook de restauration le dit lui-même : « payment-api est en mémoire : le recréer le remet dans l'état du seed » (`restore-service.yml:2`). Conséquences :

- un redémarrage (attaque A) ou une restauration (attaque B) efface **toutes** les transactions, légitimes comprises ;
- le RPO = 0 est impossible, même sans attaque ;
- la « restauration » n'est pas une restauration de données, c'est un retour à l'état de départ.

Hypothèse	Document	Verdict	Constat
Aucune transaction confirmée perdue (RPO = 0)	`cible/rto-rpo.md`	**Contredit**	API en mémoire : chaque redémarrage ou restauration perd les transactions. Aucune sauvegarde
Après une injection : snapshot puis rejeu des transactions légitimes	`cible/rto-rpo.md` §5, `cible/pca-pra.md` S2	**Contredit**	Le moteur recrée le conteneur (`restore-service.yml`) : pas de snapshot, pas de rejeu

Hypothèse	Document	Verdict	Constat
Un humain valide la restauration après une injection	`cible/recommandations.md` REC-2, `cible/pca-pra.md` S2	**Contredit**	Restauration automatique sur une seule alerte (`engine.py`)
Deux signaux concordants avant une action destructrice	`cible/pca-pra.md` S4	**Non implémenté**	Une seule alerte ML suffit. Elle combine Isolation Forest et enveloppe en « ou », ce qui n'est pas deux signaux indépendants
Au plus 3 remédiations en 10 minutes par service	`cible/pca-pra.md` §3	**Contredit**	2 tentatives par incident puis escalade (`config.py:44`, `MAX_ACTION_ROUNDS`), sans limite d'un incident à l'autre
Déni de service : limitation de débit par source, levée après 30 min	`cible/pca-pra.md` S1	**Contredit**	**Blocage complet en 403** de toute IP pesant au moins 20 % du trafic suspect (`ml/inference/service.py:36`). Pas de levée automatique : seule la remise à l'état sain débloque
Reprise vérifiée par une transaction réelle	`cible/rto-rpo.md` §1	**Confirmé**	Connexion, intention de paiement, code, paiement réussi, à travers le WAF (`services/remediation/remediation/probe.py`). Jusqu'à 6 essais espacés de 3 s
Constat d'intégrité avant restauration	`cible/pca-pra.md` S2	**Partiel**	Un simple appel à `/health`, qui répond toujours « ok »
Journaux copiés avant toute restauration	`cible/pca-pra.md` S2, §6	**Partiel**	`scripts/export-logs.sh` existe mais n'est appelé ni par le moteur ni par la remise à l'état sain
Moteur de remédiation protégé (droits limités, ordres authentifiés)	`cible/pca-pra.md` S5, `cible/recommandations.md` REC-4	**En grande partie confirmé**	Alertes signées HMAC avec contrôle de fraîcheur et anti-rejeu, liste fermée de playbooks, IP de l'hôte jamais bloquées, écoute locale. Mais il tourne sur l'hôte avec les droits Docker
Escalade vers un humain	`cible/pca-pra.md` §3	**Partiel**	Seulement après échec (événement `escalation`). Pas d'escalade systématique après une injection
Collecte toutes les 5 s, détection en 30 à 80 s	`cible/rto-rpo.md` §4	**Non mesuré**	Paramètres conformes (3 fenêtres au pas de 10 s). Le rapport montre qu'une seule requête d'injection bloquée par le WAF est détectée, mais le délai sur les attaques A et B reste à mesurer
Verrouillage progressif des comptes	`cible/ebios-rm.md` R4, `cible/recommandations.md`	**Confirmé**	2 échecs puis 5 min, durée croissante, y compris avec le bon mot de passe pendant le verrou (`InMemoryLoginAttemptTracker.ts`). Verrous en mémoire : un redémarrage les efface
Double authentification	`cible/ebios-rm.md` R4	**Confirmé pour le back-office**	Visage exigé avant le mot de passe pour les comptes enrôlés. Les comptes clients n'en ont pas
Durée de reprise affichée = RTO	`cible/rto-rpo.md` §1	**À corriger à la mesure**	Le moteur compte depuis la réception de l'alerte (T3 → T6), pas depuis le début de l'impact (T1)

Hypothèse	Document	Verdict	Constat
Assistant RAG sans accès privilégié	`cible/pca-pra.md` §5	**Confirmé**	Conteneur en lecture seule, sans droits ni accès au moteur Docker
Hôte unique, aucune sauvegarde hors de l'hôte	`cible/nis2-dora.md`, `cible/pca-pra.md` S7	**Confirmé**	Inventaire Ansible local uniquement
Secrets dans HashiCorp Vault	`cible/ebios-rm.md` BS9	**Partiellement contredit**	Vrai pour le socle et la clé HMAC (fichier hors Git). Le déploiement autonome de Pay&Go garde son mot de passe de base en clair et sa clé OpenAI en variable d'environnement

3. Nouveaux constats à intégrer à l'analyse de risques (WOR-67)

Constat	Risque concerné	Gravité du constat
Restauration automatique qui efface les transactions légitimes, déclenchée par une seule alerte ML	ER6 (remédiation en erreur), ER2 altération	**Élevée** : un faux positif d'injection suffit à perdre toutes les transactions
Blocage complet d'une IP qui pèse 20 % du trafic, sans levée automatique	ER6, R1	Moyenne : un partenaire légitime au fort volume peut être coupé jusqu'à une action manuelle
Moteur de remédiation sur l'hôte avec les droits Docker de l'utilisateur	R3, ER6	Moyenne : le bien support le plus sensible n'est pas isolé ; non mentionné dans `docs/governance/durcissement.md`
Port de la base de l'application (55432) publié sur toutes les interfaces, mot de passe en clair dans le dépôt	R5 exfiltration	**Élevée** : la base est joignable depuis le réseau `RESILIENCE-G6`
L'assistant de discussion de l'application envoie les messages à OpenAI	Fuite de données (ER3), prestataire TIC tiers (DORA art. 28)	Moyenne : données fictives, mais incohérent avec un assistant « local »
Code de vérification des paiements fixe (`123456`) **et renvoyé dans la réponse de l'API** (`demoCode`)	R4 usurpation, R2 fraude	Moyenne en démo, critique en production
Vault avec une seule part de clé, clé de descellement et jeton root dans un fichier de l'hôte	R3, R7	Moyenne : acceptable pour un prototype, à citer en limite
Attaque B sur `/api/pay`, qui n'existe pas (les routes sont en `/v1/…`)	Validité des essais	**Élevée** : l'attaque B ne touche pas l'API ; elle ne prouve que le blocage par le WAF
Pas de solde ni de débit dans le modèle de données (intentions de paiement uniquement)	ER2 altération	Faible : le risque devient « paiements altérés ou dupliqués »
Risque déjà survenu : le 21/10 entre 12 h 36 et 13 h 34, sur du trafic légitime, 2 fausses alertes d'injection ont chacune déclenché une restauration de payment-api (`ml/training/README.md`). Corrigé en réglant le détecteur	ER6, ER2	**Élevée** : confirme le constat de la première ligne. Le réglage réduit la fréquence, pas la conséquence
Une seule requête d'injection, **pourtant bloquée par le WAF**, suffit à déclencher l'alerte, donc la restauration qui efface les transactions	ER6, R2	**Élevée** : n'importe qui peut provoquer la perte des transactions avec une requête
Pendant 180 s après un redémarrage ou une restauration, le moteur ignore toute alerte	R1, R2	Moyenne : une attaque relancée dans cette fenêtre n'est pas traitée (elle est journalisée `alert_suppressed`)

Constat	Risque concerné	Gravité du constat
Verrouillage par identifiant : quelques essais au mauvais mot de passe bloquent le compte légitime, de plus en plus longtemps	R1 (disponibilité du back-office)	Faible : contournement de l'usurpation au prix d'un déni de service ciblé, classique et accepté
Le back-office répond `FACE_VERIFICATION_REQUIRED` avant de vérifier le mot de passe	R4	Faible : révèle qu'un compte existe et a un visage enrôlé
Empreintes faciales des membres de l'équipe : données biométriques **réelles** (RGPD art. 9), alors que le reste des données est fictif	ER3, RGPD	Moyenne : consentement écrit des personnes enrôlées et effacement après la démo (`DELETE /users/{user}` ou purge du volume `face-data`)
Si le service de reconnaissance faciale tombe, l'administrateur ne peut plus se connecter (erreur 502)	R1	Faible : dépendance à documenter dans le PCA (connexion de secours)

4. Ce qu'il faut avant les essais

Par ordre de priorité, pour que la démo prouve ce que la gouvernance affirme :

1. **Brancher l'API du socle sur PostgreSQL** (`STORAGE_DRIVER=postgres`) et faire un `pg_dump` avant chaque essai. Sans cela, chaque remédiation perd les transactions. Infrastructure, Développement.
2. **Évaluer le modèle sur des fenêtres d'attaque** : taux et délai de détection, faux positifs. IA / Data.
3. **Recâbler l'attaque B** sur une route en `/v1/…`. Cybersécurité.
4. **Arbitrer la restauration après injection** : automatique (code actuel) ou validée par un humain (REC-2). Au minimum, ne pas restaurer quand le WAF a bloqué toutes les requêtes d'injection. Puis aligner le code ou la gouvernance. Gouvernance, Développement.
5. **Appeler `scripts/export-logs.sh`** avant toute restauration et avant la remise à l'état sain. Développement, Infrastructure.
6. **Chronométrer T1 → T6** sur trois essais par attaque (`cible/rto-rpo.md` §6). Tous.
7. **Fermer le port 55432 et passer les secrets de l'application dans Vault.** Cybersécurité.
8. **Recueillir le consentement écrit** des personnes dont le visage est enrôlé et prévoir l'effacement après la démo. Gouvernance, Cybersécurité.